

Data Safety: Masking, Redaction, Minimization

Chapter 09 · Episode 01 · Domain 2.6 — Reduce What Exists

Upstream Beats Downstream

MINIMIZATION

Reduces what is collected

Absent data cannot leak

ENCRYPTION

Protects what remains

Applies after collection

Classification Drives Handling

Public · internal · confidential · restricted

Label training data by tier

Segment retrieval corpora at ingestion



Classify at Ingestion — Never After

Unlabeled regulated data = policy failure

Labels drive every downstream control



Masking & Redaction Techniques



STATIC

Permanent, irreversible alteration



TOKENIZE

Reversible token + separate key map



DYNAMIC

Runtime-only hiding — storage intact



REDACT

Removed entirely before ingestion

Redaction Removes — Masking Obfuscates

Static permanent · dynamic runtime-only

Tokens reversible — redaction is gone





Data Minimization

1. Collect only what the purpose requires
2. Metadata may suffice — not identifiers
3. Retention limits stop accumulation
4. Purpose-bound — no silent repurposing

Accumulation Is Exposure

Old data: stale consent, weaker controls

No retention limit = violated principle

Shorter retention → smaller breach area



Validate Data Safety Controls



AUDIT

Masking vs evolving
extraction methods



DETECT

Unclassified sensitive
data at ingestion



REVIEW

Retention + redaction
enforced in practice

Controls Claimed $\neq$ Controls Working

Drift + misconfigs silently add exposure
Minimization decides what remains at all



1 Classifier

M
O
D

2 Masking Engine

M
O
D

3 Retention Auditor

M
O
D

HOW TO RUN

WINDOWS

```
python ep28_demo.py
```

MACOS/LINUX

```
python3 ep28_demo.py
```

TRY IT Run the pipeline with dynamic masking disabled. Which identifiers surface in the output? What does this reveal about masking coverage gaps?

ep28_demo.py

pure Python 3.10+, no external libraries

Demo: Data Safety Pipeline

Colab audit — classification, masking, tokenization, retention

Episode Takeaways

Minimization reduces what exists — encryption protects what exists

Static = permanent · dynamic = runtime · token = reversible · redact = removal

Classification at ingestion — retroactive labeling fails

Governance monitoring catches drift and masking failures

Data Integrity and Output Protection

Chapter 09 · Episode 02 · Domain 2.6 — Authentic and Unaltered

Integrity vs Confidentiality

STOLEN DATASET

Serious — but the theft is visible

SILENT ALTERATION

Invisible until decisions shift

Every Transition Can Alter

Ingestion → training → inference → logs

Modified logs break forensics

Odd behavior? Suspect integrity first



Stored-Data Integrity

HASH

Any change flips
the fingerprint

IMMUTABLE

Write-once or
append-only

DETECT

Alerts on
unexpected change

VERSION

Reproducible
dataset states

Preserve original state for forensic retraining



Protect Outputs and Logs



VALIDATE

Verify outputs
before users act



SIGN

Signatures prove
origin + tampering



LOG

Append-only ·
timestamped · verifiable



HANDOFF

Every system hop
can alter output

Prove Origin and Authorship



WATERMARK

Embedded patterns verify generated output



PROVENANCE

Dataset origin + training lineage records



AUTHENTICITY

Recipients confirm a trusted source

Drift Without a Cause?

Unexpected drift may be tampering

First hypothesis: integrity compromise

Monitor continuously, not just at setup



Detection Without Restoration Fails

Roll back to verified known-good states

Alerts must trigger containment



1 Hash Verifier

M
O
D

2 Log Integrity Check

M
O
D

3 Provenance Auditor

M
O
D

HOW TO RUN

WINDOWS

```
python ep29_demo.py
```

MACOS/LINUX

```
python3 ep29_demo.py
```

TRY IT Modify one record in the training dataset and re-run the hash verifier. At what granularity is the tampering detected — file, row, or field?

ep29_demo.py

pure Python 3.10+, no external libraries

Demo: Integrity Verification

Hashing, append-only logs, signatures, provenance, rollback

Episode Takeaways

Silent modification = integrity failure — worse than theft

Hashing is the primary dataset integrity check

Logs: append-only + timestamped + verifiable

Watermark origin · provenance lineage · rollback restores

Prompt Monitoring, Logging, and Telemetry

Chapter 09 · Episode 03 · Domain 2.6 — Make Hidden Abuse Visible

Two Signals in the Telemetry

INJECTION

Repeated override-language variants

EXTRACTION

Structured high-frequency probing

Monitoring Is a Detective Control

Turns hidden abuse into visible events
Structured logging is the baseline



The Telemetry Package

PROMPT

Full or redacted content

METADATA

Tokens, size, frequency

RESPONSE

Output + confidence scores

IDENTITY

User, auth method, session

Timestamp + model version + region = forensic backbone

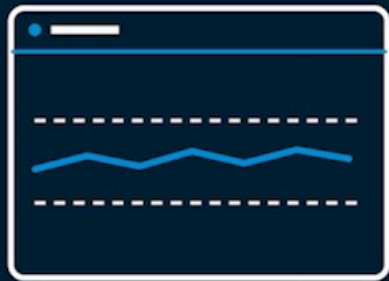


Logs Carry Sensitive Data

Protect prompt logs like source data
Investigative value vs privacy duty

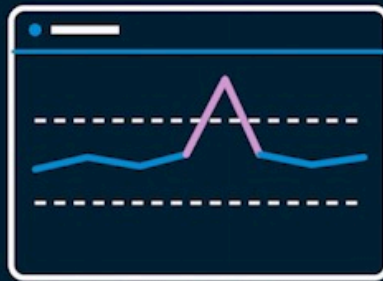


The Anomaly Detection Trio



BASELINE

Expected length, rate,
timing per role



FREQUENCY

Volume spikes →
extraction or bots



PATTERN

Repeated override
phrasing variants

Beyond Static Thresholds

Confidence swings = adversarial pressure

Cross-user ties reveal coordination

Know all three trio components



Plug AI Into the SOC



AGGREGATE
Prompt logs flow
into the SIEM



CORRELATE
Tie AI events to
identity signals



PRIORITIZE
Rank alerts — stop
analyst fatigue



RESPOND
Defined escalation
+ containment



Lawful Logging

1. Redact sensitive fields before storage
2. Retention caps — stale logs add risk
3. Access limited to authorized reviewers
4. Align with data protection law

1 Log Capture

M
O
D

2 Anomaly Detector

M
O
D

3 Privacy Redactor

M
O
D

HOW TO RUN

WINDOWS

```
python ep30_demo.py
```

MACOS/LINUX

```
python3 ep30_demo.py
```

TRY IT Disable the baseline model and use only static frequency thresholds. Which attack patterns does the system now miss?

ep30_demo.py

pure Python 3.10+, no external libraries

Demo: Live Prompt Telemetry

GPT-4o mini — capture, anomaly detection, privacy redaction

Episode Takeaways

Detective control — makes hidden abuse observable

Full package: prompt + metadata + response + identity + time

Baseline + frequency + pattern = anomaly detection

Telemetry joins the SIEM — uncontrolled logging is risk

A decorative circuit pattern in a light blue color, featuring various geometric shapes like hexagons, squares, and lines, resembling a network or data flow diagram, located in the bottom right corner of the slide.

AI Cost Monitoring and Resource Abuse

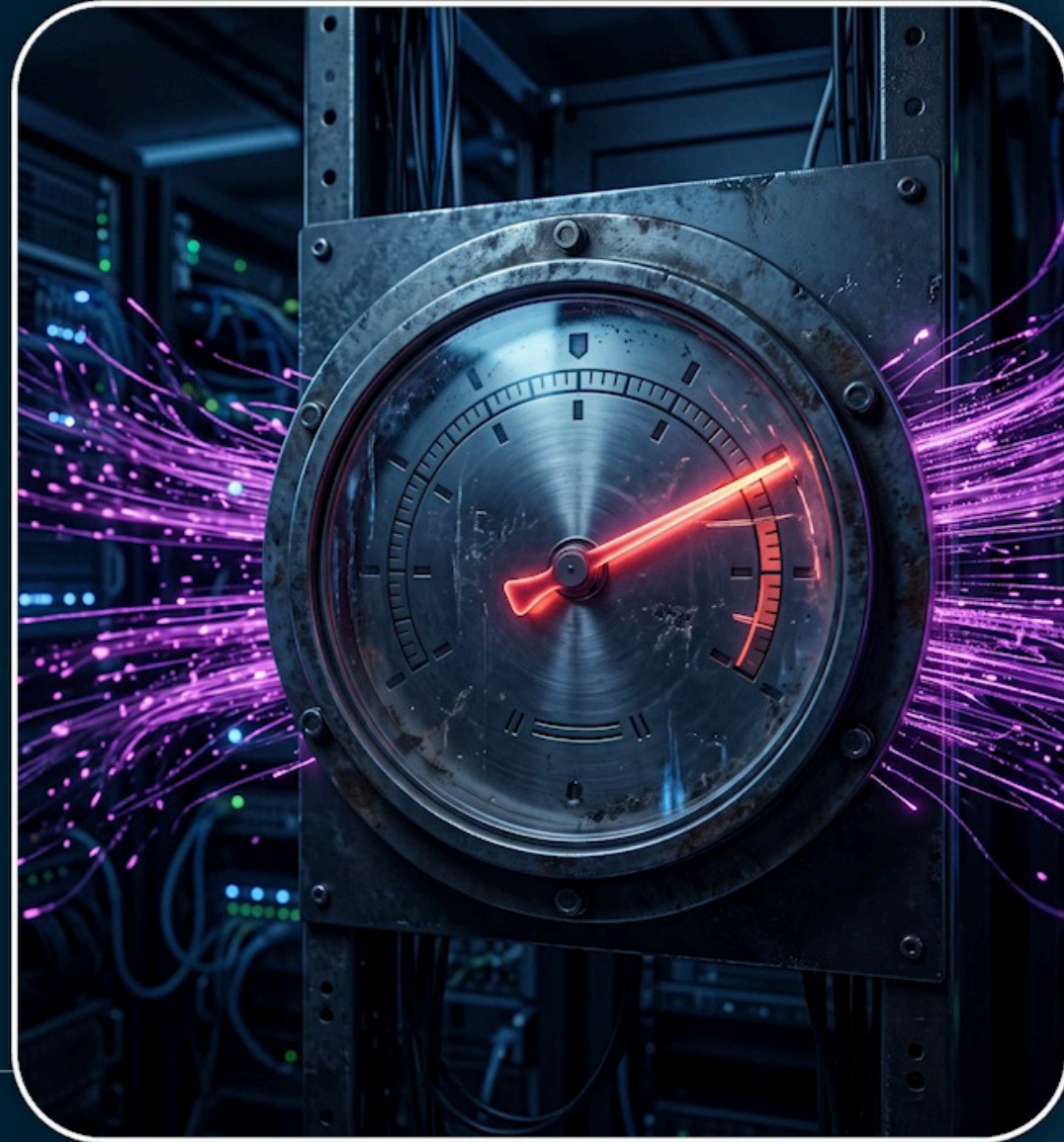
Chapter 09 · Episode 04 · Domain 2.6 — Economic Denial of Service

The Economic Attack Surface

Token billing = attackable cost curve

High-volume requests inflate spend

No breach needed — purely economic



When Cost Abuse Delays Detection

Slow AI responses, slow incident response

Cost monitoring is a security control



Token Monitoring: Two Modes

PREVENTATIVE

Ceilings block oversize requests

DETECTIVE

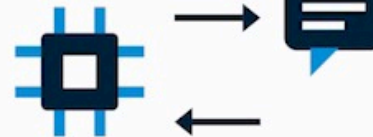
Trend analysis flags odd growth

Spotting Automated Abuse



SPEED

Machine-pace
request intervals



BOT-LIKE

Rapid follow-ups
probe prior output



FINGERPRINT

Timing, token
structure, parsing



EVASION

Volume abuse
inside rate limits

Beyond Token Counts



COMPUTE

GPU saturation flags
adversarial load



STORAGE

Dataset + embedding
growth watched



CORRELATE

Tie spikes to identities
and endpoints



Wire Alerts to Action

1. Alert thresholds trigger investigation
2. Throttle dynamically during abuse
3. Post-incident cost analysis refines policy
4. Route to security AND finance

1 Token Quota Monitor

M
O
D

2 Behavioral Profiler

M
O
D

3 Cross-Layer Correlator

M
O
D

HOW TO RUN

WINDOWS

```
python ep31_demo.py
```

MACOS/LINUX

```
python3 ep31_demo.py
```

TRY IT Simulate a bot sending 50 requests at 0.5-second intervals. Does behavioral profiling flag it before the rate limiter triggers?

`ep31_demo.py`

pure Python 3.10+, no external libraries

Demo: Resource Abuse Pipeline

Token quotas, cost anomalies, fingerprinting, correlation

Episode Takeaways

Economic DoS — real damage, no data theft

Token controls: preventative ceilings + detective trends

Behavioral detection closes the rate-limit gap

Correlate spikes to identities, not aggregates

CompTIA **SecAI+**

Quality, Bias, and Compliance Auditing In AI Systems



Monitoring vs Auditing

MONITORING

Detects misuse and attacks

AUDITING

Judges correctness + fairness

Hallucination Is an Operational Risk

Fabricated intel misleads analysts

Bias invites regulatory liability



Hallucination Detection



GROUND

Check outputs
against references



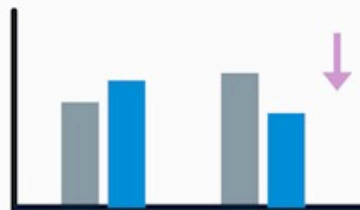
CONFIDENCE

Certainty shifts
signal drift



VERIFY

Second review on
high-impact output



REGRESS

Benchmark against
prior versions



Bias Auditing

1. Disparity analysis across groups
2. Compare FP/FN rates per category
3. Sensitivity tests on input changes
4. Documented fairness reviews

Compliance Auditing

DATA

Enforced as documented

ACCESS

Privilege reviews on record

EXPLAIN

Evidence behind decisions

TRAIL

Logs + versions inspectable

Policy claims must become verifiable evidence

CLAIMS BECOME EVIDENCE

DATA

Enforced as documented

PROOF

ACCESS

Privilege reviews on record

PROOF

EXPLAIN

Evidence behind decisions

PROOF

TRAIL

Logs + versions inspectable

PROOF



AUDIT-READY

Policy claims must become verifiable evidence.

Regulators Expect Traceability

Show how decisions were reached

Audit bridges internal + external trust



Auditing Never Ends

Scheduled reviews, cross-team committees

Incidents trigger reassessment

Findings must drive change



1 Accuracy Benchmarker

M
O
D

2 Bias Analyzer

M
O
D

3 Compliance Checker

M
O
D

HOW TO RUN

WINDOWS

```
python ep32_demo.py
```

MACOS/LINUX

```
python3 ep32_demo.py
```

TRY IT Update the model weights with a modified version and re-run the benchmarker. Which output categories show the most regression?

`ep32_demo.py`

pure Python 3.10+, no external libraries

Demo: Quality & Bias Audit

Ground truth, regression deltas, disparity, audit cycles

Episode Takeaways

Monitoring detects misuse — auditing judges correctness

Security hallucination = fabricated threat intel

Post-update inconsistency = failed regression audit

Bias is in security scope — audit continuously

CompTIA **SecAI+**

AI Attack Analysis and Evidence Evaluation



Two Ways AI Fails

OPERATIONAL

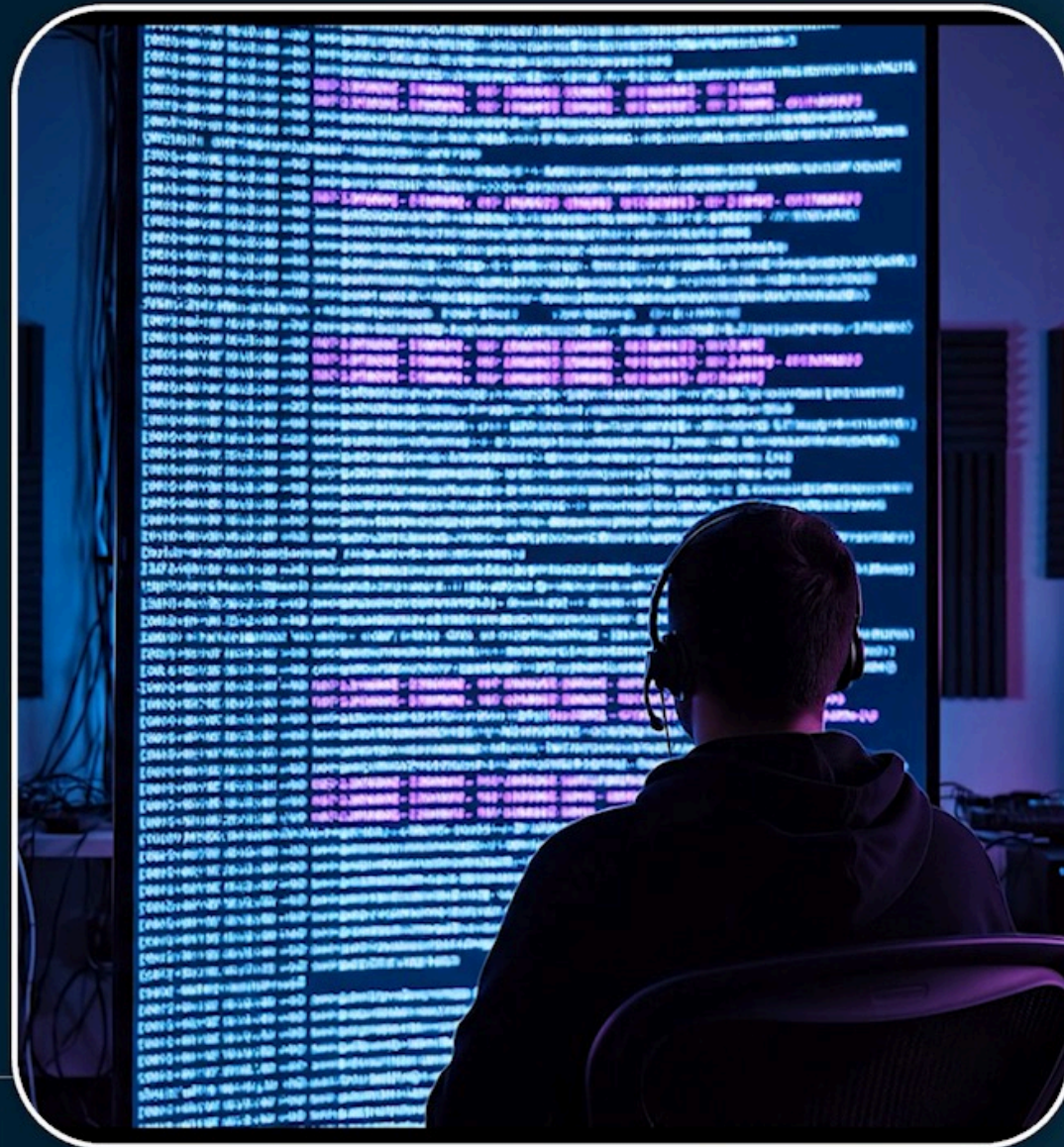
Hallucination and drift

ADVERSARIAL

Injection · poisoning · extraction

Reading Injection Evidence

Override language in prompt logs
Correlate identity: insider or external
Guardrails: triggered or bypassed?



Crafted Input + Intact Data = Injection

Causality: which control failed first?

Target compensating controls precisely

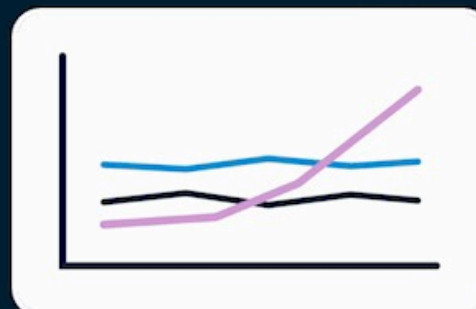


Poisoning: Follow the Dataset



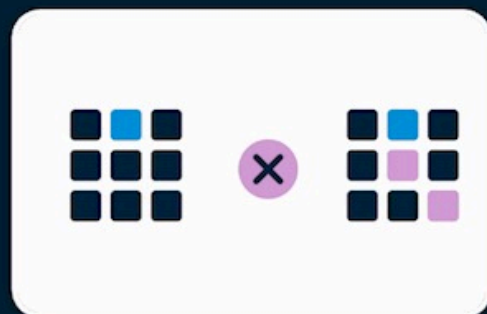
COMPARE

Current vs prior
validated versions



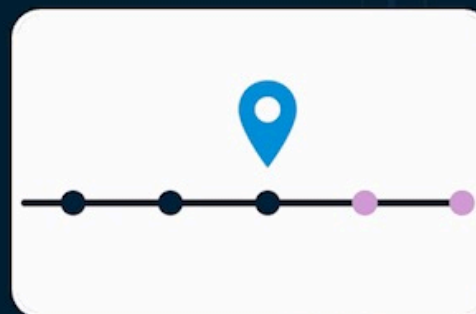
DRIFT

Targeted misclassification patterns



HASH

Mismatch signals
integrity failure



TIMELINE

Did anomaly follow retraining?

Extraction: Sustained Recon

Incremental high-frequency probing
Unusual token efficiency per request
Rate limits: triggered or bypassed?



Match Control to Vector

Injection → firewall + segmentation

Poisoning → validation + ingestion

Wrong control = testable wrong answer



1 Injection Evidence

S
C
N

2 Poisoning Evidence

S
C
N

3 Extraction Evidence

S
C
N

HOW TO RUN

WINDOWS

```
python ep33_demo.py
```

MACOS/LINUX

```
python3 ep33_demo.py
```

TRY IT Present a symptom (unexpected model bias after retraining). Walk through the evidence checklist to determine injection vs poisoning vs drift – before reading the answer.

`ep33_demo.py`

pure Python 3.10+, no external libraries

Demo: Attack Analysis

Incident signals — logs, hashes, telemetry — classified

Episode Takeaways

Layer first — it picks the control category

Injection: override logs + intact dataset

Poisoning: misclassification + hash mismatch + audit trail

Extraction: probing + token + rate anomalies

Domain 2 Capstone: Defense in Depth

Chapter 09 · Episode 07 · Domain 2.6 — Layered Remediation

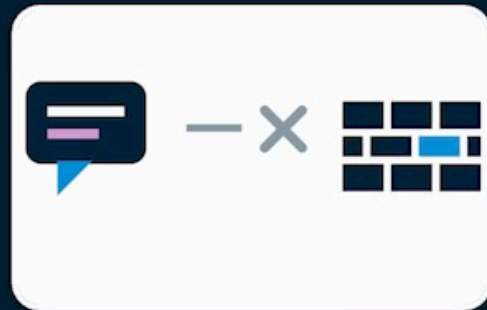
Precision Beats Piling On Controls

Threat → control → layered remediation

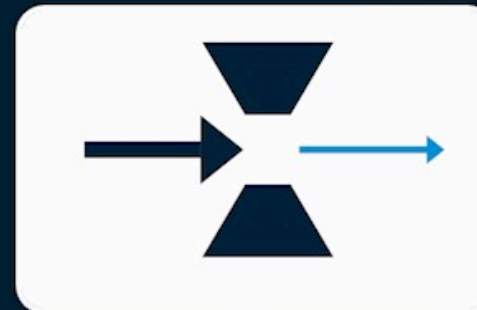
Maturity = correct mapping, not volume



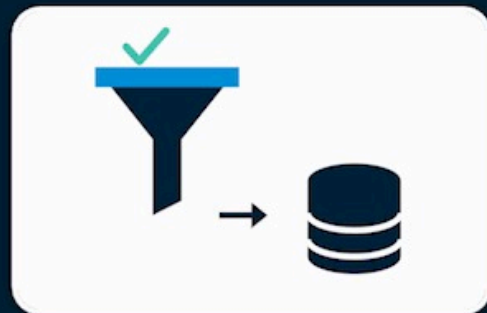
Attack-to-Control Mapping



INJECTION
Prompt firewall
+ segmentation



EXTRACTION
Rate limits
+ output restriction



POISONING
Dataset validation
+ ingestion



AGENCY
Tool scoping + HITL
+ timed tokens

Compensating Controls Buy Time

Backstop when primary controls slip
HITL while firewall rules mature
Temporary — never the architecture



Assume a Layer Will Fail

No single control is infallible

Resilience comes from the stack



Controls at Every Layer



DATA

Encryption,
masking, hashing



MODEL

Evaluation +
adversarial training



INTERFACE

Firewalls, token +
rate limits, auth



INFRA

Segmentation, service
auth, monitoring



Prioritize by Risk

1. Impact $\times$ likelihood guides effort
2. High-value assets protected first
3. Capacity goes to consequential exposure
4. Reassess as threats evolve

Domain 2, Assembled

Modeling → taxonomy → rails → access

Encryption, data safety, monitoring

One coordinated, layered architecture



1 Injection incident

S
C
N

2 Poisoning incident

S
C
N

3 Extraction incident

S
C
N

HOW TO RUN

WINDOWS

```
python ep34_demo.py
```

MACOS/LINUX

```
python3 ep34_demo.py
```

TRY IT For each scenario: identify the layer, name the primary control, name one compensating control, and identify one misaligned control that would be a wrong exam answer.

`ep34_demo.py`

pure Python 3.10+, no external libraries

Demo: Defense in Depth Drill

Scenario → vector + layer → gaps → compensating controls

Episode Takeaways

Vector-to-control mapping is the top-tested skill

Misaligned remediation = testable wrong answer

Compensating controls buy time, not safety

Layer → control category → layered remediation